Informatics II, Spring 2026, Solution 7

Task 1

1.

push(4)	push(1)	push(3)	pop()	push(8)	pop()
		3		8	
	1	1	1	1	1
4	4	4	4	4	4

2.

enqueue(4)	enqueue(1)	enqueue(3)	dequeue()	enqueue(8)	dequeue()
		3		8	
	1	1	3	3	8
4	4	4	1	1	3

3. The first stack starts at 1 and grows up towards n, while the second starts from n and decreases to 1. Stack overflow happens when an element is pushed when the two stack pointers are adjacent.

4. ENQUEUE: $\Theta(1)$.
DEQUEUE: worst $O(n)$, amortized $\Theta(1)$(on average).
Let the two stacks be A and B.
ENQUEUE pushes elements on B. ENQUEUE is always $\Theta(1)$. DEQUEUE pops elements from A. If A is empty, the contents of B are transferred to A by popping them one by one out of B and pushing them to A until B is empty. That way, they appear in reverse order and are popped in the original order.
DEQUEUE operation can perform in $\Theta(n)$ time, but that will happen only when A is empty. If many ENQUEUEs and DEQUEUEs are performed, the total time will be linear to the number of elements. For example, we ENQUEUE n elements and DEQUEUE n elements. All n elements are transferred from B to A only once, so in total n times. The amortized complexity of DEQUEUE = n / n = 1, which is $\Theta(1)$(on average).

5. PUSH: $\Theta(1)$.
POP: $\Theta(n)$.
We have two queues – q_1 and q_2. PUSH operation always enqueues elements in q_1. Assume that q_1 contains i elements: e_1,...,e_i. POP operation: (1) dequeue e_1,...,e_{i-1} elements and remain element e in q_1 (2) enqueue e_1,...,e_{i-1} in order to q_2. (3) dequeue e from q_1 and return e.
The PUSH operation is $\Theta(1)$. The POP operation is $\Theta(n)$ where n is the number of elements in the stack. In other words, there are n elements in q_1.

Task 2.1

```c
#include <stdio.h>
#define SIZE 10

int stack[SIZE];
int top = -1;

void push(int value){
    if(top<SIZE-1){
        if (top < 0){
            stack[0] = value;
            top = 0;
        }
        else{
            stack[top+1] = value;
            top++;
        }
    }
    else{
        printf("Stackoverflow!!!!\n");
    }
}

int isempty() {
    return top<0;
}

int pop(){
    if(!isempty()){
        int n = stack[top];
        top--;
        return n;
    }
    else{
        printf("Error: the stack is empty!\n");
        return -99999;
    }
}

int Top(){
    if (!isempty()){
        return stack[top];
    }
    else{
        printf("Error: the stack is empty!\n");
        return -99999;
    }
}

void display(){
    int i;
    for(i=0;i<=top;i++){
        printf("%d,",stack[i]);
    }
    printf("\n");
}

int main(){
    push(4);
    push(8);
    printf("isempty: %d\n", isempty());
    printf("Top: %d\n", Top());
    display();

    pop();
    printf("\nisempty: %d\n", isempty());
    printf("Top: %d\n", Top());
    display();

    pop();
    printf("\nisempty: %d\n", isempty());
    printf("Top: %d\n", Top());
    display();

    pop();

    return 0;
}
```

Task 2.2

```c
#include <stdio.h>
#define MAXSIZE 10

int queue[MAXSIZE];

int front = -1;
int rear = -1;
int size = -1;

int isempty()
{
    return size <= 0;
}

int isfull()
{
    return size == MAXSIZE;
}

void enqueue(int value)
{
    if(size<MAXSIZE)
    {
        if(isempty())
        {
            queue[0] = value;
            front = rear = 0;
            size = 1;
        }
        else if(rear == MAXSIZE-1)
        {
            queue[0] = value;
            rear = 0;
            size++;
        }
        else
        {
            queue[rear+1] = value;
            rear++;
            size++;
        }
    }
    else
    {
        printf("Queue␣is␣full\n");
    }
}

int Front()
{
    if(isempty())
    {
        printf("Queue␣is␣empty\n");
        return -1;
    }
    else
    {
        return queue[front];
    }
}

int dequeue()
{
    int ret = Front();
    size--;
    front++;
    if (front == MAXSIZE) {
        front = 0;
    }
    return ret;
}

void display()
{
    if(isempty())
    {
        printf("Queue␣is␣empty\n");
        return;
    }

    int i;
    if(rear>=front)
    {
        for(i=front;i<=rear;i++)
        {
            printf("%d,",queue[i]);
        }
    }
    else
    {
        for(i=front;i<MAXSIZE;i++)
        {
            printf("%d,",queue[i]);
        }
        for(i=0;i<=rear;i++)
        {
            printf("%d,",queue[i]);
        }
    }
        printf("\n");
}

int main()
{
    display();
    enqueue(4);
    enqueue(8);
    enqueue(10);
    enqueue(20);
    display();
    dequeue();
    printf("After␣dequeue\n");
    display();
    enqueue(50);
    enqueue(60);
    enqueue(70);
    enqueue(80);
    dequeue();
    enqueue(90);
    enqueue(100);
    enqueue(110);
    enqueue(120);
    enqueue(130);
    enqueue(140);
    enqueue(150);
    printf("After␣enqueue\n");
    display();
    dequeue();
    printf("After␣dequeue\n");
    display();
    enqueue(160);
    printf("After␣enqueue\n");
    display();
    return 0;
}
```

Task 2.3

```c
#include <stdio.h>
#include <stdlib.h>
#define TRUE 1
#define FALSE 0

struct node
{
    int data;
    struct node *next;
};
typedef struct node node;

node *top;

void initialize()
{
    top = NULL;
}

void push(int value)
{
    node *tmp;
    tmp = malloc(sizeof(node));
    tmp -> data = value;
    tmp -> next = top;
    top = tmp;
}

int pop()
{
    node *tmp;
    int n;
    tmp = top;
    n = tmp->data;
    top = top->next;
    free(tmp);
    return n;
}

int Top()
{
    return top->data;
}

int isempty()
{
    return top==NULL;
}

void display(node *head)
{
    if(head == NULL)
    {
        printf("NULL\n");
    }
    else
    {
        printf("%d,", head -> data);
        display(head->next);
    }
}

int main()
{
    initialize();
    push(10);
    push(20);
    push(30);
    printf("The top is %d\n",Top());
    pop();
    printf("The top after pop is %d\n",Top());
    display(top);
    return 0;
}
```

Task 2.4

```c
#include <stdio.h>
#include <stdlib.h>
#define TRUE 1
#define FALSE 0
#define FULL 10

struct node
{
    int data;
    struct node *next;
};
typedef struct node node;

struct queue
{
    int count;
    node *front;
    node *rear;
};
typedef struct queue queue;

void initialize(queue *q)
{
    q->count = 0;
    q->front = NULL;
    q->rear = NULL;
}

int isempty(queue *q)
{
    return (q->rear == NULL);
}

void enqueue(queue *q, int value)
{
    if (q->count < FULL)
    {
        node *tmp;
        tmp = malloc(sizeof(node));
        tmp->data = value;
        tmp->next = NULL;
        if(!isempty(q))
        {
            q->rear->next = tmp;
            q->rear = tmp;
        }
        else
        {
            q->front = q->rear = tmp;
        }
            q->count++;
    }
    else
    {
        printf("List␣is␣full\n");
    }
}

int dequeue(queue *q)
{
    node *tmp;
    int n = q->front->data;
    tmp = q->front;
    q->front = q->front->next;
    if(q->front == NULL)
        q->rear = NULL;
    q->count--;
    free(tmp);
    return(n);
}

void display(node *head)
{
    if(head == NULL)
    {
        printf("NULL\n");
    }
    else
    {
        printf("%d,", head -> data);
        display(head->next);
    }
}

int main()
{
    queue *q;
    q = malloc(sizeof(queue));
    initialize(q);
    enqueue(q,10);
    enqueue(q,20);
    enqueue(q,30);
    printf("Queue␣before␣dequeue\n");
    display(q->front);
    dequeue(q);
    printf("Queue␣after␣dequeue\n");
    display(q->front);
    return 0;
}
```

Task 3.

Algorithm: reverseEven(Q)

```
S = initStack()
qSize = queueSize(Q)
for i = 1 to qSize do
    val = deQueue(Q)
    if val %2 ==0 then
        push(S,val)
    enQueue(Q,val)
for i = 1 to qSize do
    val = deQueue(Q)
    if val %2 ==0 then
        enQueue(Q,pop(S))
    else
        enQueue(Q,val)
```